

**INSPT - Universidad Tecnológica Nacional**

Programación I — Comisión 1.605

Docentes: Behringer Alejandro Jorge | Capia Juan Carlos

**Trabajo Práctico N° 03****Sentencias if — Operadores Relacionales y Lógicos**

|                             |                                                         |
|-----------------------------|---------------------------------------------------------|
| <b>Trabajo Práctico N°:</b> | 03                                                      |
| <b>Fecha:</b>               | 21 de abril de 2026                                     |
| <b>Tema:</b>                | Sentencias if — Operadores Relacionales y Lógicos       |
| <b>Unidades:</b>            | U3 — Programación Estructurada (Estructuras Selectivas) |
| <b>Modalidad:</b>           | Trabajo practico de clase                               |
| <b>Lenguaje:</b>            | C (estándar ANSI C / C99)                               |

**Instrucciones generales**

- Resolvé cada ejercicio en un archivo .c separado (ej: tp03\_ej01.c, tp03\_ej02.c).
- Incluí siempre: `#include <stdio.h>`, `#include <math.h>` y `#include "utiles.h"`.
- Usá `leerEntero()` o `leerFlotante()` de `utiles.h` para la lectura de datos.
- Declará rangos y constantes con `#define` (sin magic numbers).
- La estructura de control principal es `if / else if / else`. No uses `switch` en este TP.
- Prohibido: `while(1)`, `break` para salir de bucles, `continue`, `fflush(stdin)`.
- El programa debe finalizar con `tecla()` antes del `return 0`.
- Mostrá siempre los datos ingresados antes de mostrar el resultado.

**Ejercicio 01 — Mayor de dos números**

El programa solicita al usuario dos números enteros y determina cuál es el mayor. Antes de operar, valida que ninguno sea negativo.

**Datos de entrada:**

- Primer número entero (ingresado por el usuario)
- Segundo número entero (ingresado por el usuario)

**Lógica para implementar:**

```
Si alguno de los números es negativo:  
    → Mostrar mensaje de error y finalizar el programa
```

```
Si ambos son no negativos:  
    Guardar el mayor en una variable llamada mayor  
    → Mostrar cuál de los dos es el mayor
```

**Salida esperada:** *Mostrar los dos números ingresados e indicar cuál es el mayor (o si son iguales).*

## Ejercicio 02 — Verificación de rango

El programa solicita un número entero al usuario y verifica si se encuentra dentro del rango válido.

### Datos de entrada:

- Un número entero cualquiera (ingresado por el usuario)

### Lógica por implementar:

```
Si 1 <= número <= 100 → "El número está DENTRO del rango [1, 100]"
Caso contrario          → "El número está FUERA del rango [1, 100]"
```

**Salida esperada:** *Mostrar el número ingresado y el mensaje de rango correspondiente.*

## Ejercicio 03 — Año bisiesto

El programa solicita un año al usuario y determina si es bisiesto o no, aplicando las reglas del calendario gregoriano.

### Datos de entrada:

- Un año entero positivo (ingresado por el usuario)

### Reglas del año bisiesto:

```
Es bisiesto si:
    (divisible por 4 Y NO divisible por 100)
    O bien
    (divisible por 400)

En cualquier otro caso → NO es bisiesto
```

**Salida esperada:** *Mostrar el año ingresado e indicar si es bisiesto o no.*

## Ejercicio 04 — Calificación de examen

El programa solicita la calificación numérica de un examen (número entero entre 0 y 100) y asigna la letra correspondiente. Se deben usar if anidados.

### Datos de entrada:

- Calificación entera entre 0 y 100 (validar el rango)

### Escala de calificaciones:

```
A → 90 a 100    (ej: 95)
B → 80 a 89     (ej: 85)
C → 70 a 79     (ej: 75)
D → 60 a 69     (ej: 65)
F → 0 a 59      (ej: 50)
```

**Salida esperada:** *Mostrar la calificación numérica y la letra asignada (A / B / C / D / F).*

## Ejercicio 05 — Ordenamiento de tres números

El programa solicita tres números enteros al usuario y los muestra ordenados de menor a mayor, usando únicamente sentencias if y operadores relacionales (sin vectores ni funciones de ordenamiento).

**Datos de entrada:**

- Tres números enteros (A, B y C), ingresados por el usuario

**Lógica a implementar:**

```
Usando comparaciones con if / else if:
    Determinar el orden correcto de A, B y C
    Mostrar los tres valores de menor a mayor
```

**Salida esperada:** *Mostrar los tres números ingresados y luego los tres ordenados de menor a mayor.*

**Ejercicio 06 — Ecuación de segundo grado**

El programa resuelve una ecuación cuadrática de la forma  $ax^2 + bx + c = 0$  a partir de los coeficientes ingresados por el usuario. La naturaleza de las raíces se determina con el discriminante.

**Datos de entrada:**

- Coeficiente a (término cuadrático) — debe ser distinto de cero
- Coeficiente b (término lineal)
- Coeficiente c (término independiente)

**Fórmulas:**

```
Discriminante:   Delta = b^2 - 4 * a * c

x1 = ( -b + sqrt(Delta) ) / ( 2 * a )
x2 = ( -b - sqrt(Delta) ) / ( 2 * a )
```

**Casos según el discriminante:**

```
Delta > 0 → Dos raíces reales distintas: mostrar x1 y x2
Delta = 0 → Una raíz real doble: mostrar x1 (= x2)
Delta < 0 → Raíces complejas: avisar al usuario, no calcular
```

**Salida esperada:** *Mostrar los coeficientes ingresados, el valor del discriminante y las raíces (o el aviso de raíces complejas).*

**Ejercicio 07 — Conversión de unidades: centímetros a pies y pulgadas**

El programa recibe una medida en centímetros y la convierte a pies (enteros) y pulgadas (con un decimal).

**Datos de entrada:**

- Medida en centímetros (número real, tipo float — debe ser positiva)

**Factores de conversión:**

```
1 pulgada = 2.54 cm
1 pie      = 12 pulgadas

pulgadas_totales = centimetros / 2.54
pies              = (int) pulgadas_totales / 12      ← parte entera
pulgadas_resto   = pulgadas_totales - (pies * 12)   ← con 1 decimal
```

**Salida esperada:** *Mostrar los centímetros ingresados, los pies (entero) y las pulgadas restantes (1 decimal).*

**Ejercicio 08 — Días vividos**

El programa solicita la edad del usuario en años y calcula el número aproximado de días que ha vivido.

**Datos de entrada:**

- Edad en años enteros (debe ser positiva y razonable — validar rango)

**Fórmula:**

```
dias_vividos = edad * 365
```

**Nota:** Para este ejercicio se usa el año de 365 días (sin considerar años bisiestos).

**Salida esperada:** Mostrar la edad ingresada y el total de días vividos.

## Ejercicio 09 — Distancia entre dos puntos — Logística de entregas

Una empresa de logística representa su ciudad en un plano cartesiano donde cada unidad equivale a un kilómetro. El programa calcula la distancia en kilómetros entre el almacén principal (Punto A) y el cliente final (Punto B).

**Datos de entrada:**

- Coordenadas del Punto A: (x1, y1) — valores reales
- Coordenadas del Punto B: (x2, y2) — valores reales

*Ejemplo de referencia:* Punto A = (3, 5) | Punto B = (10, 12)

**Fórmula de la distancia euclidiana:**

```
distancia = sqrt( (x2 - x1)² + (y2 - y1)² )
```

En C:

```
distancia = sqrt( pow(x2 - x1, 2) + pow(y2 - y1, 2) )
```

**Salida esperada:** Mostrar las coordenadas ingresadas y la distancia en kilómetros con dos decimales.

## Criterios de evaluación

| Criterio                                                           | Ponderación |
|--------------------------------------------------------------------|-------------|
| Correctitud lógica de las condiciones y resultados                 | 35 %        |
| Uso correcto de if / else if / else (sin switch)                   | 20 %        |
| Estándares de la cátedra (utils.h, #define, sin fflush, sin break) | 20 %        |
| Validación adecuada de los datos de entrada                        | 10 %        |
| Nombres descriptivos y legibilidad del código                      | 10 %        |
| Comentarios pertinentes                                            | 5 %         |